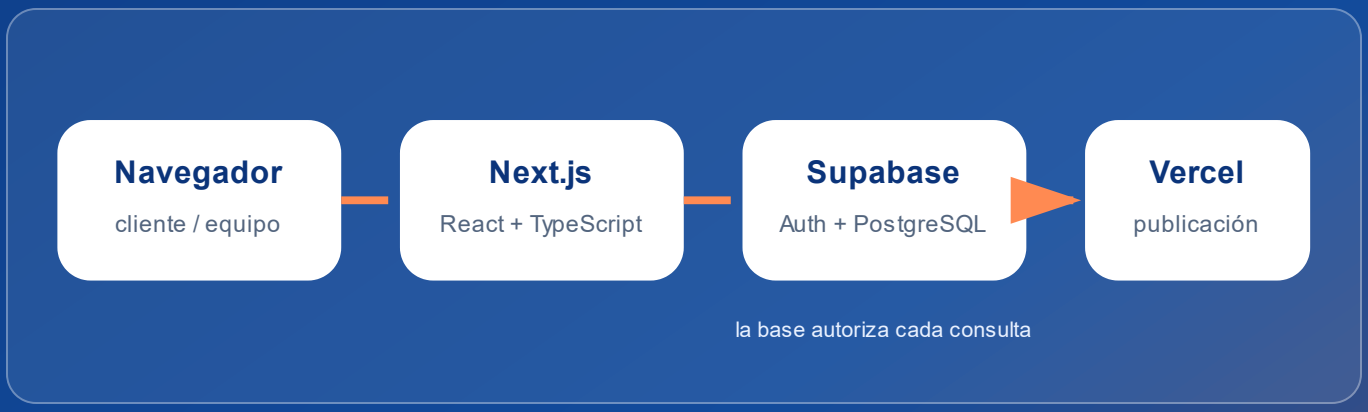
APRENDE CONSTRUYÉNDOLA · 2026

Cómo se hizo ChamoEntregas

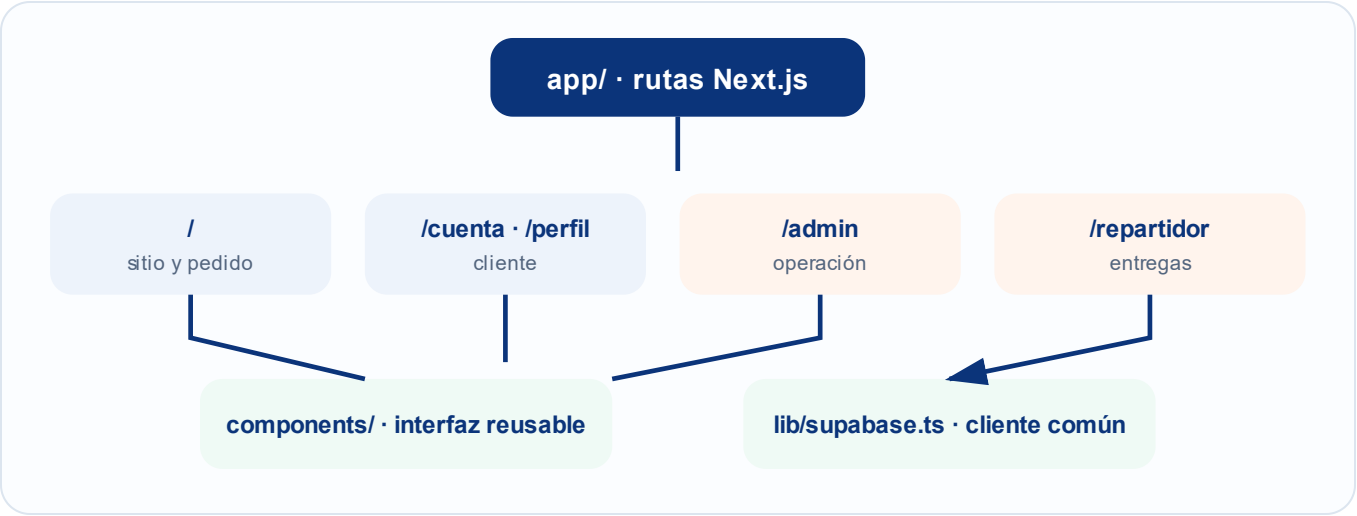
Un tutorial de principio a fin: interfaz, cuentas, base de datos, permisos, operación, reputación y publicación web.



Hecho para Coloncito, Táchira · Sitio: chamoentregas.vercel.app

Qué aporta cada pieza

La aplicación se divide para que cada responsabilidad tenga un lugar claro. El navegador muestra pantallas; Supabase autentica usuarios y protege los datos; Vercel sirve la versión pública.



Archivo o carpeta	Responsabilidad
`app/`	URL y páginas que Next.js entrega.
`components/`	Pantallas y controles React; separa la lógica visual por función.
`lib/demo-data.ts`	Tipos compartidos, estados y datos de muestra.
`lib/supabase.ts`	Construye un cliente Supabase reutilizable con las variables públicas.
`supabase/schema.sql` y `migrations/`	Definen tablas, políticas RLS, funciones y cambios graduales de base.

02 · PREPARAR Y EJECUTAR

Primeros pasos en el computador

1. 1 Instala Node.js LTS y Git.
2. 2 Descarga el repositorio y abre su carpeta:

```
git clone https://github.com/frankbuitrago3-code/chamoentregas.git
cd chamoentregas
npm install
```

3. 3 Copia `.env.example` como `.env.local`.
4. 4 Completa la URL y la clave publicable del proyecto Supabase:

```
NEXT_PUBLIC_SUPABASE_URL=https://TU-REF.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=TU-CLAVE-PUBLICABLE
```

5. 5 Inicia el servidor y abre el navegador:

```
npm run dev
# http://localhost:3000
```

Vista pública:
`/`

Cuenta del cliente:
`/cuenta` y `/perfil`

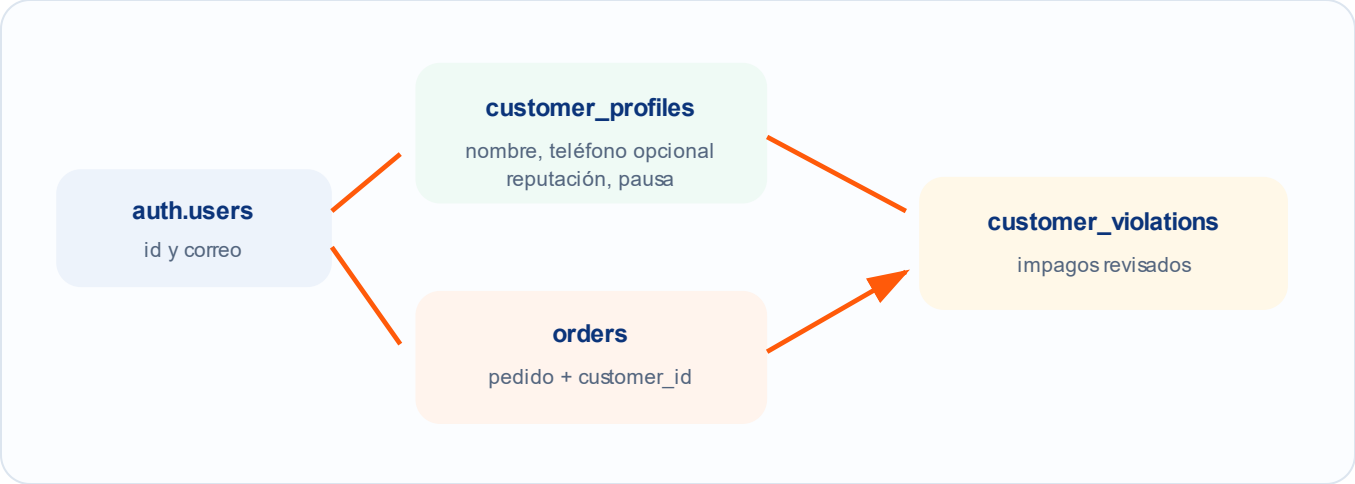
Operación:
`/admin`

Repartidor:
`/repartidor`

Nunca subas `.env.local`, contraseñas ni la clave `service_role` a GitHub. El navegador solo recibe la clave publicable; RLS hace cumplir los permisos.

Supabase conecta autenticación con perfiles

PostgreSQL guarda la información. Supabase Auth guarda credenciales y crea una identidad UUID. Una función de base de datos crea automáticamente la fila de perfil correspondiente.

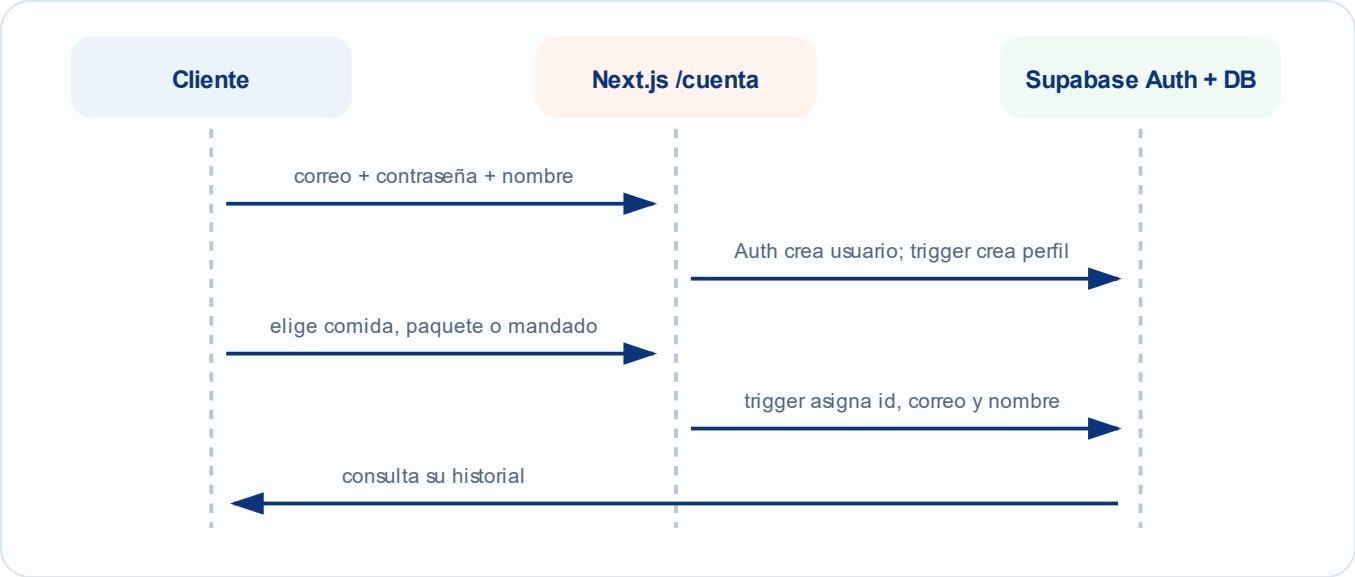


Instalar o actualizar la base

1. En Supabase abre **SQL Editor**.
2. Para una base nueva, ejecuta `supabase/schema.sql`.
3. Ejecuta la migración `supabase/migrations/20261015_customer_accounts_and_reputation.sql`. En esta app ya creada, corre esa migración una sola vez.
4. Comprueba que existen `customer_profiles`, `customer_violations` y las nuevas columnas de `orders`.

> La migración crea el perfil a partir de usuarios Auth existentes y también configura un trigger para próximos registros. No insertes credenciales directamente en tablas de negocio.

Por qué la cuenta y la base trabajan juntas



Archivos que implementan el recorrido

Archivo	Qué hace
`components/customer-account.tsx`	Registra e inicia sesión con Supabase Auth; pide nombre y email, el teléfono no aparece como requisito.
`components/chamo-landing.tsx`	Lee la sesión y el perfil, asocia el pedido al UUID de la cuenta y deja opcional el teléfono.
`components/customer-profile.tsx`	Permite editar nombre/teléfono y leer solo pedidos e incidencias propios.
`set_order_customer_identity()`	En servidor reemplaza identidad enviada por navegador con la de la sesión y rechaza cuentas pausadas.

```
signUp({ email, password, options: { data: { full_name } } })
orders.insert({ type, city, destination, ... })
-- El trigger completa customer_id y customer_email.
```

RLS es la puerta de seguridad de los datos

RLS significa Row Level Security (seguridad por fila). Aunque el navegador tenga una clave publicable, cada consulta a la base debe cumplir reglas antes de leer o escribir.

Cliente autenticado
Puede enviar pedidos nuevos y ver los de su propio `customer_id`; actualiza solo nombre y teléfono.

Administrador
Se comprueba `app_metadata.role = admin`; gestiona pedidos y revisa clientes.

Repartidor
Ve pedidos nuevos de su ciudad y los asignados a su perfil activo.

Visitante anónimo
No puede enviar pedidos: debe registrarse o entrar primero.

Cómo opera el equipo

1. El cliente envía pedido; la base lo registra como `nuevos`.
2. Admin revisa datos, cliente y cobertura; asigna a un repartidor.
3. Repartidor acepta y actualiza recogida, camino y entrega.
4. Admin observa la tabla actualizada por Realtime y coordina incidencias.



> Nunca intentes “proteger” datos ocultando botones solamente. La autorización final debe estar en RLS o en una función SQL con verificación de rol.

Decisiones humanas, reglas automáticas seguras

La aplicación automatiza el registro y el cálculo, no decide si alguien actuó mal. Una persona administradora revisa primero el caso.

1 · Identificar
Cliente y pedido entregado vinculado.

2 · Revisar
Administrador explica la evidencia comprobada.

3 · Registrar
Función protegida crea un evento único.

4 · Reflejar
Trigger resta 20 puntos, con mínimo 0.

Qué está en el código SQL

- `customer_violations` guarda pedido, cliente, evidencia resumida, nota interna, fecha y administrador.
- La llave única sobre pedido y tipo evita un descuento duplicado por la misma entrega.
- `admin_confirm_customer_nonpayment()` verifica el rol y comprueba que el pedido esté entregado y pertenezca al usuario.
- Un trigger actualiza la reputación dentro de la transacción, así registro y cálculo no se separan.
- `admin_set_customer_order_block()` permite pausar nuevas solicitudes 1, 3 o 7 días; la puntuación por sí sola no bloquea.

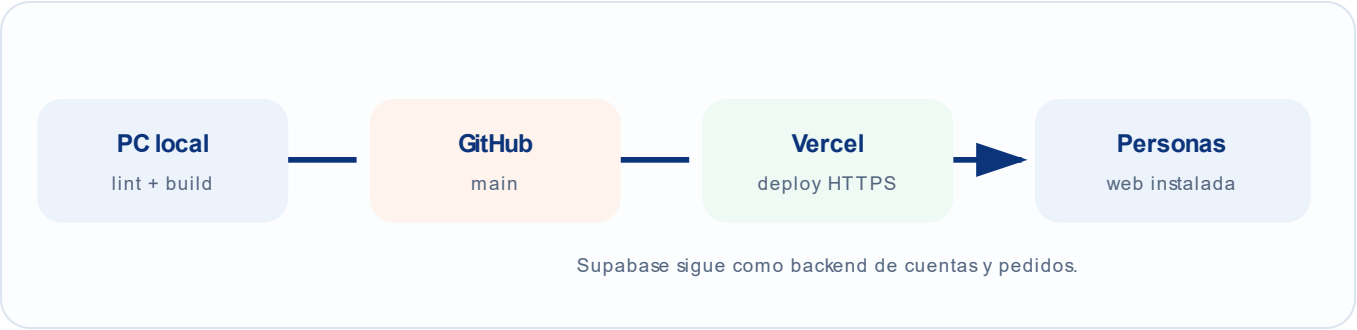
```
100 puntos iniciales
- 20 por cada impago confirmado por admin
mínimo: 0
sin bloqueo automático

Ejemplo: 100 → 80 → 60
El admin puede limitar 3 días tras revisar el caso.
```

Por qué así: evita que un repartidor o un visitante pueda acusar y sancionar directamente; se conserva el motivo y el cliente puede verlo en su perfil y contactar a soporte.

De los cambios locales a la web pública

1. **Desarrollo:** cambiar componentes y SQL en la rama local; usar datos ficticios al probar.
2. **Base de datos:** revisar migración y RLS; aplicar el SQL al proyecto Supabase correcto.
3. **Calidad:** `npm run lint` y `npm run build`.
4. **Control de versiones:** confirmar solo los archivos previstos y publicar el commit en GitHub.
5. **Despliegue:** Vercel obtiene el commit, usa las variables de entorno y publica HTTPS.
6. **Prueba real:** abrir cuenta cliente, pedir con teléfono vacío, verificar historial, revisar panel y estados.



Checklist de aceptación

- La cuenta se crea y el correo se confirma si la configuración lo requiere.
- Una solicitud sin iniciar sesión se rechaza; una cuenta válida sí puede enviar pedido.
- Nombre y email se asignan desde Auth; teléfono puede quedar vacío.
- El perfil solo permite consultar pedidos propios.
- El panel registra un impago solo para pedido entregado con evidencia; el segundo reporte del mismo pedido falla.

Cómo seguir aprendiendo y ampliar la app

Para cambiar una pantalla
Busca la página en `app/`; la interfaz suele estar en su componente de `components/`.

Para cambiar un dato
Actualiza esquema/migración, tipos TypeScript y los formularios o consultas que lo usan.

Para proteger una acción
Añade política RLS o función SQL, prueba rol permitido y rol rechazado.

Para publicar
Revisa lint/build, revisa diff, sube a GitHub y comprueba el despliegue de Vercel.

Funciones futuras que requieren diseño adicional

- Subida real de fotos a Storage con permisos privados y políticas de acceso.
- Pago en línea, reembolsos y conciliación, si se decide abandonar efectivo.
- GPS en vivo y consentimiento/retención de ubicación.
- Apelaciones, reglas de recuperación de reputación y proceso de revisión de disputas.

> Antes de añadir una función, define quién puede usarla, qué datos exige, quién puede leerlos, qué ocurre cuando falla y cómo se prueba.

Proyecto: github.com/frankbuitrago3-code/chamoentregas
Producción: chamoentregas.vercel.app

Material de aprendizaje vinculado al código actual del proyecto. No incluye secretos ni contraseñas.